EduTrack — HLD & LLD

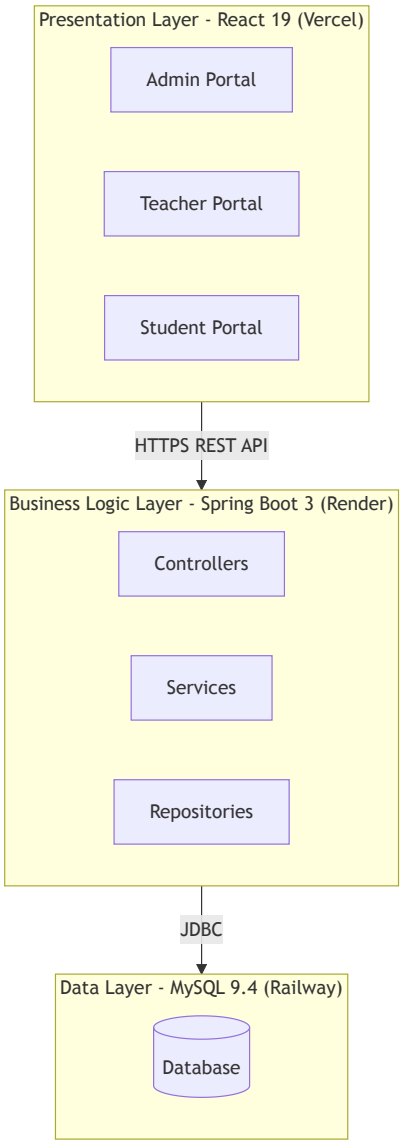
Academic Management Platform

VTU Internship 2026 — Team 15

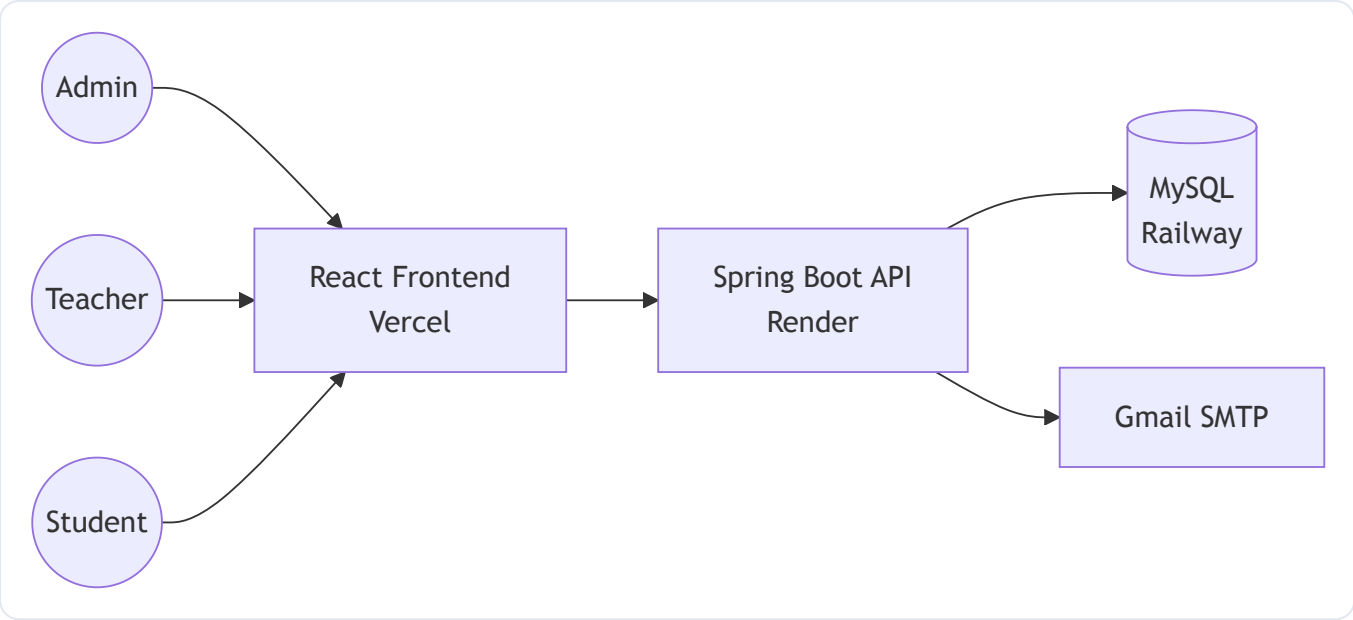
PART 1 — HIGH LEVEL DESIGN (HLD)

1. System Architecture

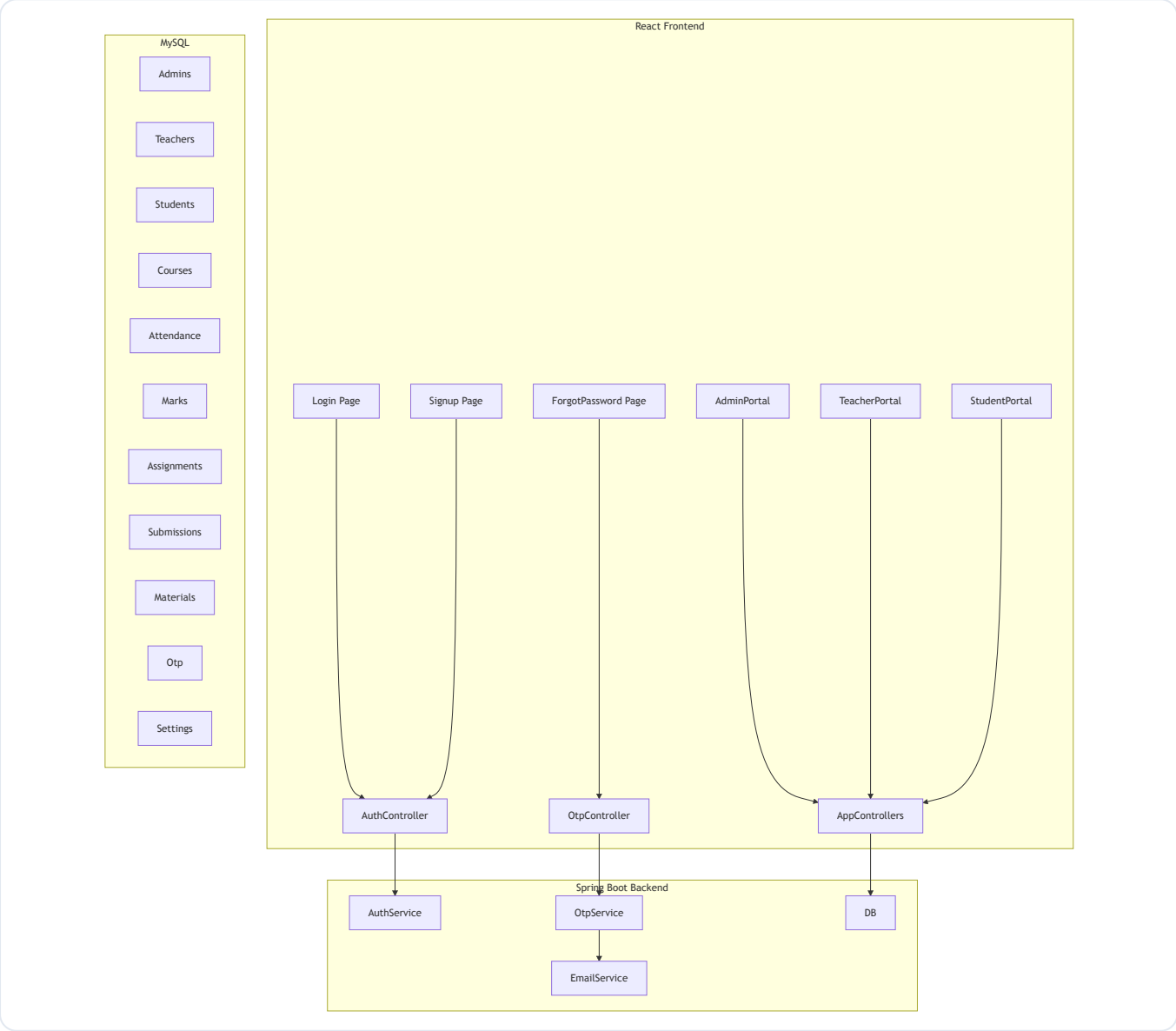
EduTrack follows a 3-tier architecture:



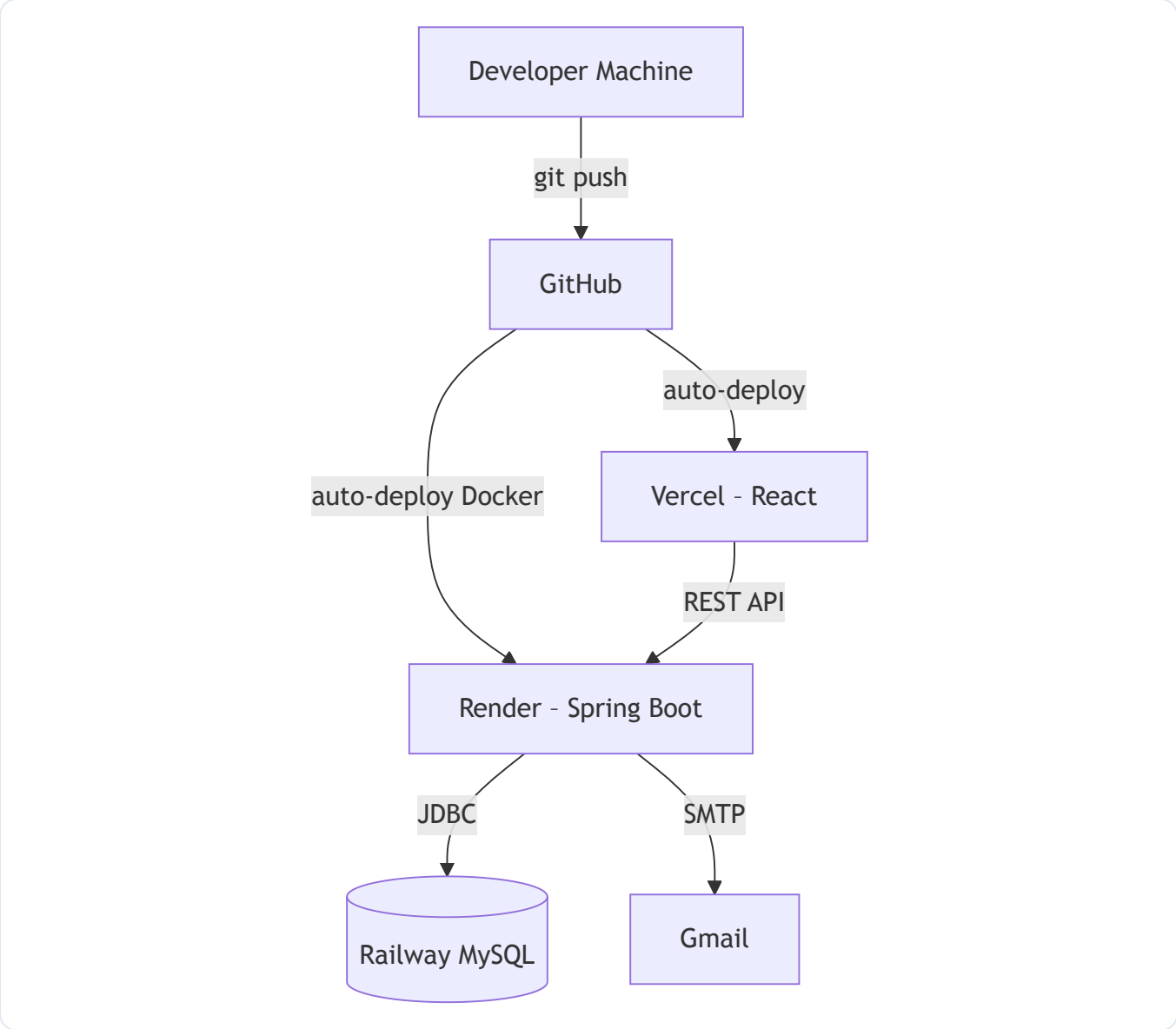
2. System Context Diagram



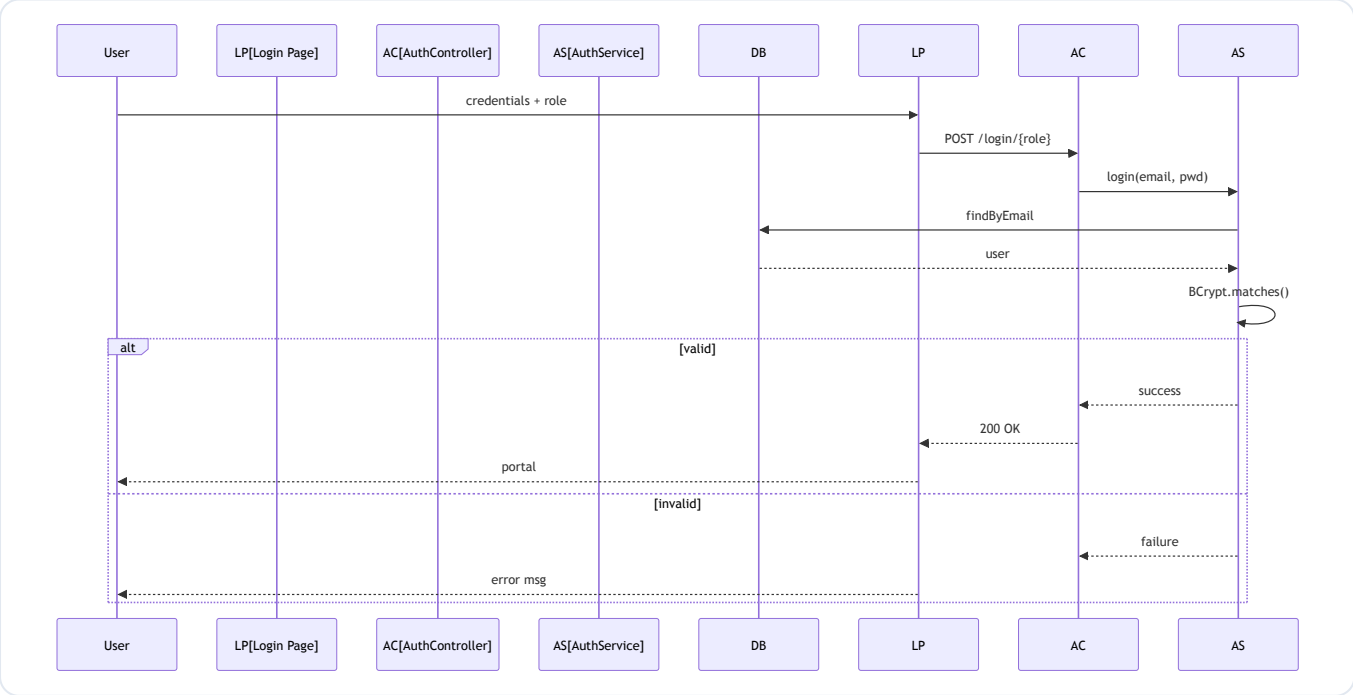
3. Component Diagram



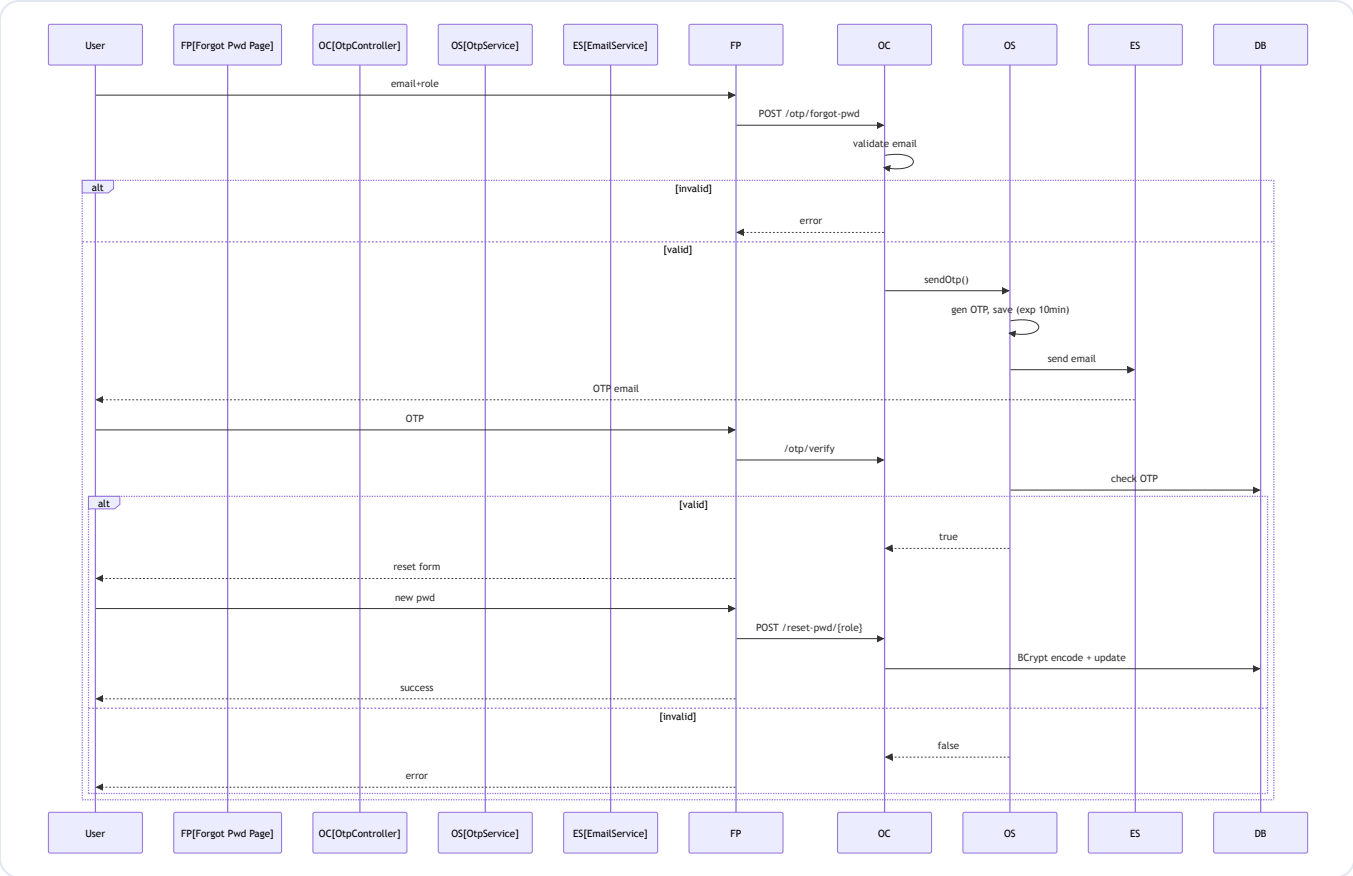
4. Deployment Diagram



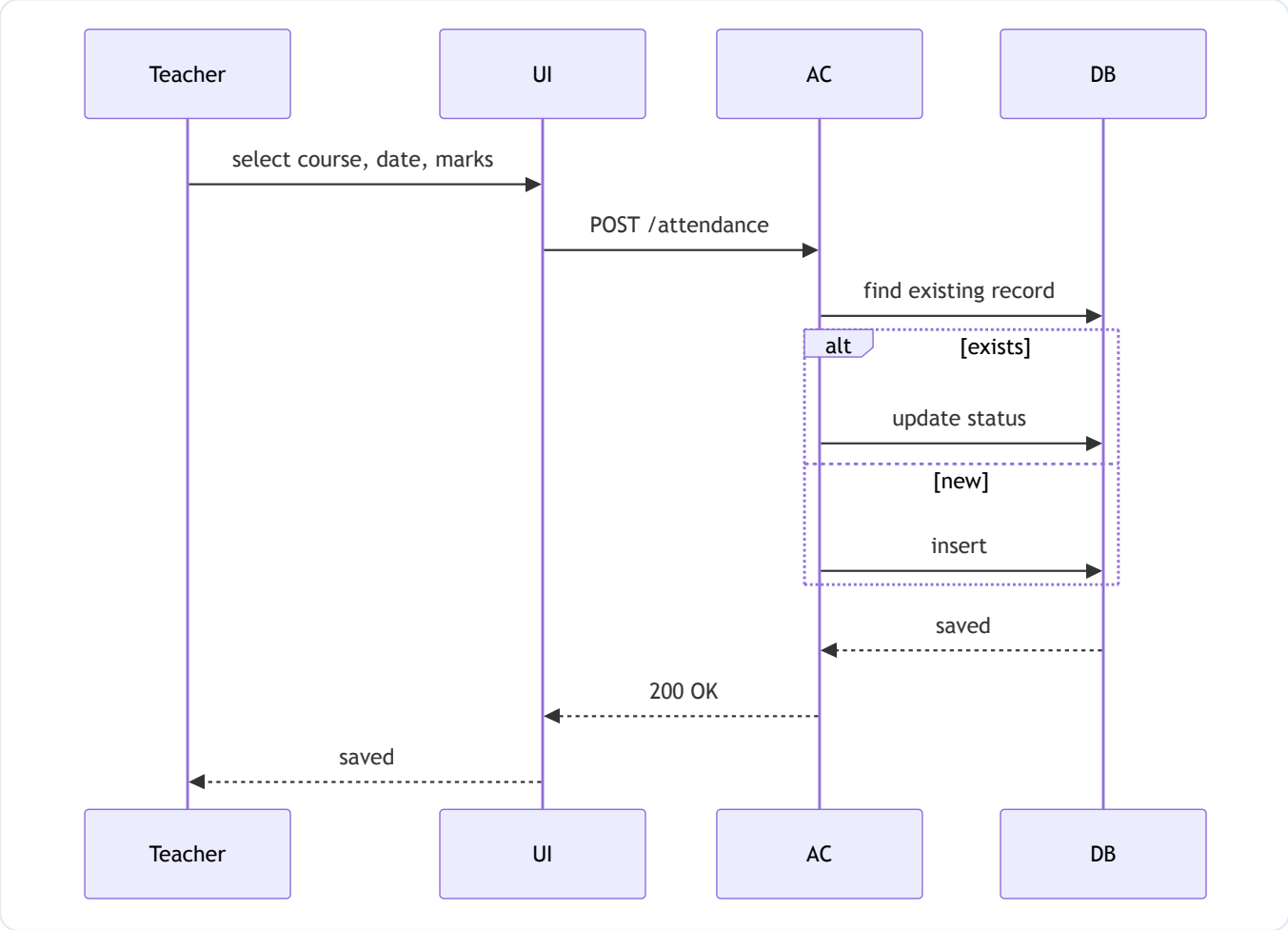
5. Authentication Flow



6. OTP Reset Flow

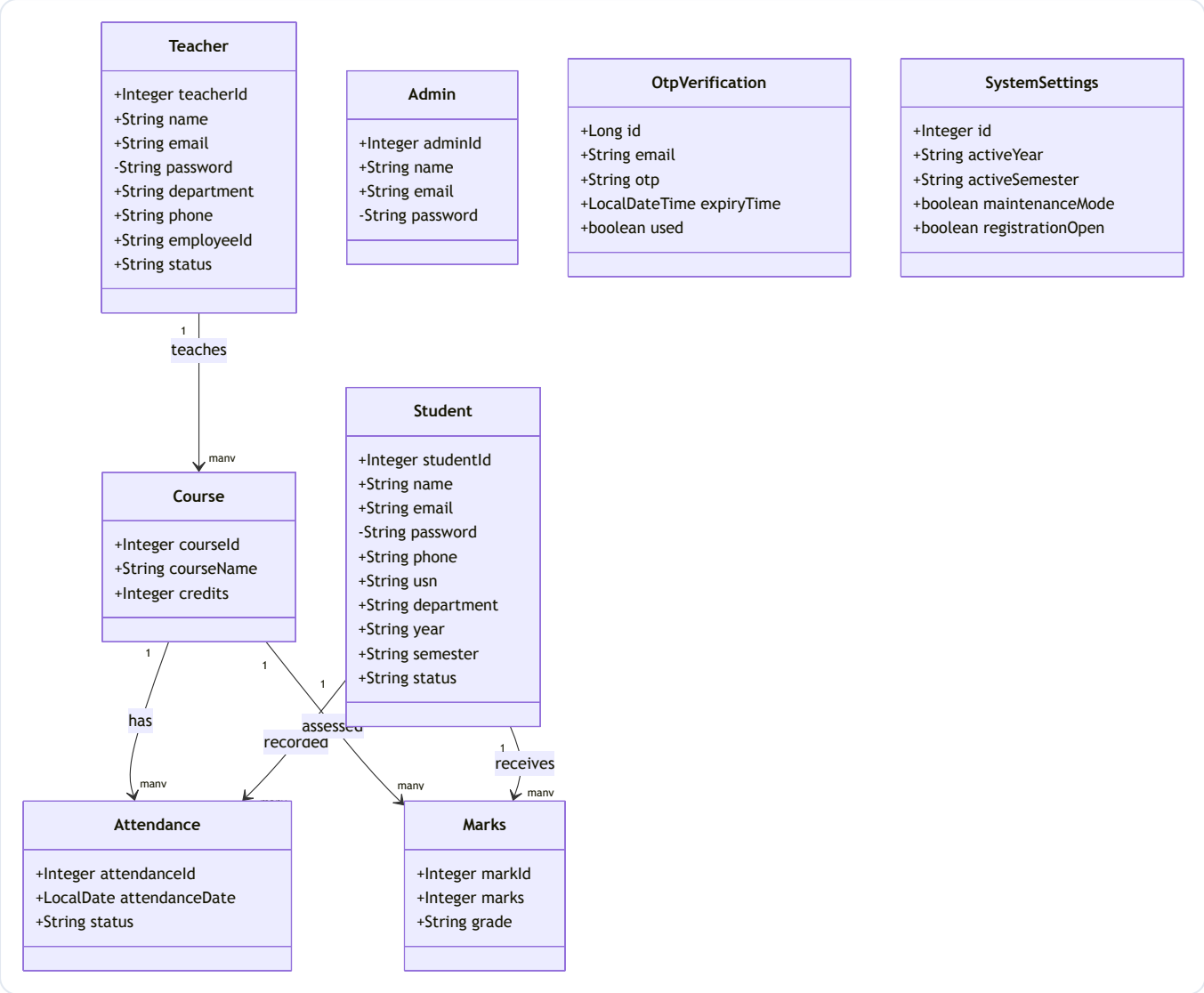


7. Mark Attendance



PART 2 — LOW LEVEL DESIGN (LLD)

8. Entity Class Diagram



9. Correct Entity Code

9.1 Student Entity (Actual)

```
@Entity
@Table(name = "students")
@Data @NoArgsConstructor @AllArgsConstructor
public class Student {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer studentId;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    @Column(nullable = false)
    private String password;

    private String phone;
    @Column(unique = true) private String usn;
    @Column(nullable = false) private String department;
    @Column(nullable = false) private String year;
    private String semester;
    @Column(nullable = false) private String status = "Active";

    @ManyToOne @JoinColumn(name = "course_id")
    private Course course;
}
```

9.2 Teacher Entity (Actual)

```
@Entity
@Table(name = "teachers")
@Data @NoArgsConstructor @AllArgsConstructor
public class Teacher {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer teacherId;

    @Column(nullable = false) private String name;
    @Column(nullable = false, unique = true) private String email;
    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    @Column(nullable = false) private String password;
    @Column(nullable = false) private String department;
    private String phone;
    @Column(unique = true) private String employeeId;
    @Column(nullable = false) private String status = "Active";
}
```

9.3 Attendance Entity (Actual)

```
@Entity
@Table(name = "attendance", uniqueConstraints =
    @UniqueConstraint(columnNames =
{"student_id","course_id","attendance_date"}))
@Data @NoArgsConstructor @AllArgsConstructor
public class Attendance {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer attendanceId;

    @ManyToOne @JoinColumn(name = "student_id", nullable = false)
    private Student student;
    @ManyToOne @JoinColumn(name = "course_id", nullable = false)
    private Course course;
    @Column(nullable = false) private LocalDate attendanceDate;
    @Column(nullable = false) private String status; // Present or Absent
}
```

10. Service Method Design

10.1 AuthService.loginAdmin()

```
Input: email, password
1. adminRepo.findByEmail(email)
2. If empty or password mismatch → return {success:false}
3. Return {success:true, adminId, name, email, role:"admin"}
```

10.2 OtpService.validateOtp()

```
1. findTopByEmailOrderByIdDesc(email)
2. if absent → false
3. if used → false
4. if expired → false
5. if otp mismatch → false
6. set used=true, save → return true
```

10.3 MarksController.enterMarks()

```
Grade: >=90 → O, >=80 → A+, >=70 → A, >=60 → B+, else B
```

11. Repository Methods

```
StudentRepository: findByEmail, existsByEmail, findByUsn,
findByDepartmentIgnoreCase
AttendanceRepository: findByStudent_StudentId, findByCourse_CourseId,
    findByCourse_CourseIdAndAttendanceDate, findByStudent+Course+Date,
deleteBy...
OtpRepository: findTopByEmailOrderByIdDesc, deleteByEmail
```

12. Security Configuration

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.csrf(csrf -> csrf.disable())
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
    return http.build();
}
CORS: allowedOriginPatterns("*"), methods GET,POST,PUT,DELETE,OPTIONS
```